

Agentic Workflows:

1. Introduction to Agentic Workflows:

2. Understanding Agentic Workflows

The Modern AI Agent: A System with a "Brain"

Modern AI agents use a Large Language Models (LLMs) as their central processing unit, or 'brain'. This is what allows agents to understand goals, reason about complex situations, and make decisions.

But it's more than just a brain. A true agent is a complete system designed to:

- Perceive its environment.
- Reason about what it observes.
- Formulate plans to achieve its goals.
- Execute actions to carry out those plans.

This ability to perceive, reason, plan, and act with a high degree of autonomy is what separates these agents from simple automation.

Agentic Workflow or Simple Automation?

Where do we draw the line between an agentic workflow and standard automation?

If a system just follows a series of sequential, fixed steps, we're talking about automation, not an agentic workflow. A perfect example is the automated email drip campaign:

1. A user subscribes to a newsletter.
2. The system automatically sends a pre-written "Welcome Email."
3. After 3 days, it automatically sends a pre-written "Follow-up Email."

This is a classic deterministic workflow. It's efficient for its one specific task, but it cannot adapt or make decisions.

So, what's the difference in an agent-based system? Instead of having prescribed steps to execute, the agents must decide which steps to take, and how to execute the details of each distinct process they are presented.

The AI and LLMs are the technology that enables this decision-making, but the core concept is this shift from a fixed script to a flexible, goal-oriented

process. Now, let's break down the components that give AI agents this powerful capability.

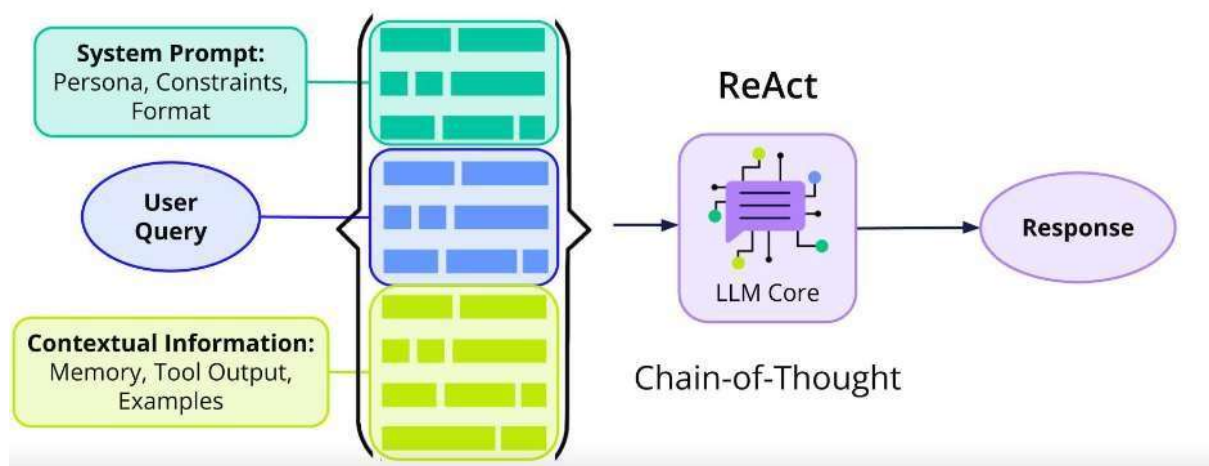
Deconstructing the AI Agent

Now that we've established that an agent is the critical component that makes a workflow "agentic," let's look at what an agent is made of. To understand how an agent can dynamically make decisions, we need to understand its anatomy.

To truly understand agents, we need to look at their core components. Think of these as the building blocks that give an agent its capabilities.

1. **Persona:** This defines the agent's identity and role.
2. **Knowledge:** This is all the information an agent can access, including the LLM's training data, fine-tuning, external tools, and memory.
3. **Prompting Strategy:** This is the blueprint for how the agent communicates with its core LLM, including system prompts and advanced techniques like "Chain-of-Thought".
4. **Execution/Tools:** This is what allows an agent to act by accessing APIs, running code, or interacting with other systems.
5. **Interaction:** This component defines how the agent communicates with the outside world, be it users or other agents.

Let's explore each of these five components and see how they contribute to an agent's function within a workflow.



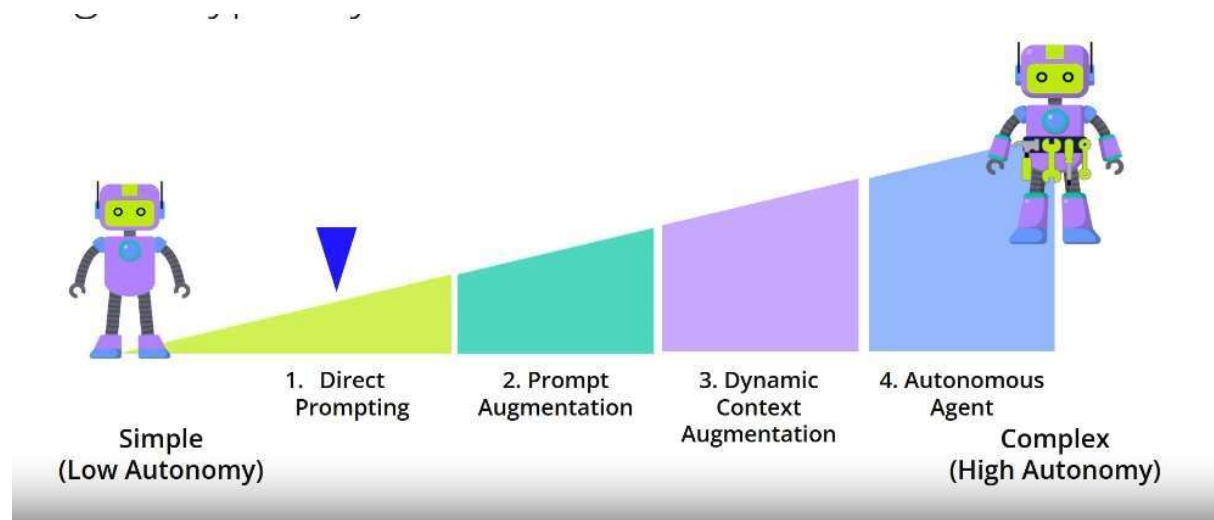
- **Persona:** The Persona defines the agent's specific role in the workflow. Think of it like giving your agent a job title. Is it a "Formal Business Analyst" or a "Friendly Customer Support Bot"? This persona, which we typically set using system prompts, guides the agent on:
 - Its specific function.
 - The appropriate tone and style for its communications.
 - The boundaries of its expertise.
 - The format for its outputs (like JSON or bullet points), so the next step in the workflow can understand them. Essentially, the persona focuses the LLM's broad capabilities into a consistent identity suitable for a specific task within our workflow.
- **Knowledge:** An agent's Knowledge is all the information it can access to do its job. It's much more than the LLM's base training. For our workflows, this includes:
 - **Foundational LLM Training:** The extensive knowledge embedded in the model.
 - **Fine-tuning:** Specific knowledge we can add by training the model on specialized data.
 - **External Information via Tools:** This is critical for dynamic workflows. It's information an agent retrieves on-the-fly from databases, APIs, or web searches. This keeps the agent's knowledge current.
 - **Memory:** Information from current or past interactions, allowing the agent to maintain context within a workflow. This combination allows our agents to make decisions grounded in relevant, specific, and up-to-date information.
- **Prompting Strategy:** The Prompting Strategy is our blueprint for instructing the agent. It's how we tell the agent what its task is within the workflow. This includes:
 - **System Prompts:** The foundational instructions that define the agent's persona and overall goal.

- Context Incorporation: How we weave information from memory or from previous steps in the workflow into the current prompt.
- Prompt Engineering Techniques: Using methods like Chain-of-Thought, ReACT and providing few-shot examples to guide the LLM's reasoning process.
- Structuring Instructions: Clearly defining objectives and the required format for the output, ensuring it's usable for the next stage of the workflow. A good prompting strategy is key to getting reliable and wellreasoned results from our agents.
- Execution/Tools: The Execution/Tools component gives our agent its hands and feet—the ability to act on behalf of the workflow. Tools allow an agent to:
 - Interact with the external world, like accessing real-time information from a weather API.
 - Manipulate data by running calculations or querying a database. □ Connect with other systems to send emails or update records.
 - Orchestrate tasks by calling upon other specialized agents. This execution capability is what transforms an agent from a simple chatbot into an active system that can move a workflow forward.
- Interaction: Finally, the Interaction Component defines how the agent communicates and exchanges information with the workflow. This covers:
 - Receiving Input: How the agent gets its instructions and data for a specific task.
 - Delivering Output: How the agent provides its results back to the workflow, whether as text, structured JSON, or by triggering another action.
 - Inter-Agent Communication: The protocols agents use to talk to each other, which is essential for building complex, multi-agent workflows. This component ensures our agent can e ectively connect with its operational environment, whether that involves humans or other software agents.

Agent Types and Their Role in Workflows

We've established that agents are the core components that bring intelligence to our workflows. However, not all agents are built the same, and not all workflows have the same requirements. The way an agent interacts with its LLM "brain" directly determines its capabilities and, therefore, the complexity of the workflow it can support.

It is helpful to think of these agent types as a spectrum of capabilities. On one end, we have simple, direct interactions suitable for straightforward, single-step workflows. As we move along the spectrum, the agents become more sophisticated, enabling more complex, adaptive, and multi-step agentic workflows.



Agent Types by LLM Interaction

Agent Type	LLM Role	Context Handling	Autonomy Level	Typical Use Case
Direct Prompting	Responder/ Generator	Static (within prompt)	Low	Simple Q&A, basic text generation
Prompt Augmentation	Guided Responder/ Generator	Pre-augmented static context (e.g. RAG)	Low-Medium	RAG-based Q&A, few shot tasks
Dynamic Context Augmentation	Adaptive Responder Processor	Dynamically Updated via memory/tools	Medium	Multi-Turn conversations, tool use
Autonomous Agent	Planner/ Executer/ Reasoner	Dynamic Internal/External memory-driven	High	Complex Task Completion, independent problem solving

- Direct Prompting Agents:
- Description: This is the most basic agent type. It acts as a direct relay, sending a user's prompt to the LLM and returning the response without modification.
- Role in a Workflow: This agent is best suited for simple, single-task workflows. For example, a workflow designed only to answer a user's question or generate text from a single instruction would use a Direct Prompting Agent.
- Prompt Augmentation Agents:
- Description: This agent enhances the prompt before sending it to the LLM. It "augments" the user's input with additional context, such as a predefined persona, few-shot examples, or retrieved documents.
- Role in a Workflow: This type allows us to create specialized steps within a workflow. A workflow could employ a "Financial Analyst" agent that uses specific financial data to augment its prompts, ensuring its outputs are contextually relevant and distinct from, for instance, a "Creative Writer" agent in the same workflow.
- Dynamic Context Augmentation Agents:
- Description: This is a more advanced agent that can adapt during an interaction. It uses tools (like a web search or API call) and memory to update the LLM's context in real-time.
- Role in a Workflow: This agent is the key to building dynamic, multi-step workflows. A workflow step powered by this agent can perform an action (e.g., search for real-time stock prices), process the result, and use that new information to inform the very next action (e.g., make a trade recommendation), all within a single, continuous process.
- Autonomous Agents:
- Description: At the most advanced end of the spectrum, these agents leverage the LLM for complex planning, decision-making, and multi-step execution with minimal human intervention.
- Role in a Workflow: These agents act as the orchestrators or primary executors of the most sophisticated agentic workflows.

An autonomous agent can take a high-level goal, independently create a plan of subtasks, execute that plan using various tools, and even self-correct if it encounters errors, driving the entire workflow from start to fi

3. Agentic Workflows

Agentic Workflow Modeling

Let's explore Agentic Workflow Modeling. Think of it as learning how to draw the blueprints for a team of AI "agents" that work together to get complex jobs done. Our goal is to understand how to design these workflows so they're not only powerful in what they can achieve but also flexible enough to adapt when things change.

- How to think about and plan these agent-based systems.
- Why focusing on the agents themselves (an "agent-centric" approach) is important.
- How to visually map out what each agent does and how they connect.

From Simple Automation to Smart Agentic Workflows

So, how do we even start thinking about designing an agentic workflow?

1. Starting with What You Know: Deterministic Automation

- Often, a good first step is to look at a process that's already welldocumented and perhaps even automated in a simple, step-by-step way. This is called deterministic automation – it follows a fixed script, like a basic email auto-responder or a simple data processing routine.
- When you first try to use AI agents for such a process, you might just replace each step with an agent. The result? Still pretty deterministic. The agents do what they're told, in the order they're told. And that's perfectly okay as a starting point!

2. The Key Leap: Generalization

- The real power of agentic workflows comes when we generalize this initial automation. If all you need is to execute one specific,

unchanging process efficiently, deterministic automation might be all you need.

- But what if you want that system to handle more types of tasks, or to find smarter, more efficient ways to get the job done without you reprogramming every detail? That's when you move towards an agentic workflow.
- This "generalization" involves making your system more intelligent. For example, you might add a "planning agent" at the beginning. This agent could look at a new request, figure out what needs to be done, and then decide which other agents or tools are needed to accomplish the goal, much like a smart project manager. Example: The Evolving Code Assistant Imagine a simple code assistant designed only to answer one specific type of Python syntax question (deterministic automation). To make it truly agentic and more useful, you could generalize it. You might give it:
 - A planning capability: to understand different types of coding questions (Python, Java, debugging, new features).
 - Access to tools: like a knowledge base of code examples, a code validation tool, or even the ability to search the web for solutions.
 - Decision-making power: to decide whether to answer from its knowledge, use a tool, or ask you for more clarification. This generalized assistant can now handle a much broader range of tasks and adapt its approach, transforming from a fixed-process follower into an intelligent, adaptive system. This path from specific automation to a generalized agentic workflow is a common and practical way to develop these systems.

Workflows vs. Chatbots: A Quick Distinction

It's important to remember we're building workflows designed to accomplish specific tasks or processes. These workflows have a defined start and end. This is different from a continuous conversational chatbot, where the interaction is ongoing until the user decides to stop. Agentic workflows are more about completing a job.

However, this doesn't mean they're completely rigid. A key strength of agentic workflows is their ability to:

- Reflect and iterate: Agents within the workflow can review their own work or the work of others to improve the outcome.
- Find different paths: They aren't always stuck on one track; they can choose different sequences of actions to achieve a goal, potentially finding more efficient methods than a strictly deterministic system.

Modeling: Process-Focused vs. Agent-Focused

How you design or "model" these workflows also changes:

- Deterministic Workflow Modeling (Process-Centric):
 - This is straightforward. You identify all tasks, map their sequence and dependencies, define inputs/outputs for each, note decision points (if any, they are usually simple rules), and visualize it all (like a traditional flowchart). The focus is on the rigid connection between tasks.
- Agentic Workflow Modeling (Agent-Centric):
 - Here, the focus shifts to the agents themselves. You define:
 - Agent capabilities: What can each agent do?
 - Goal structure: What are the objectives, and how is success measured?
 - Decision-making frameworks: How do agents evaluate options and make choices?
 - Adaptation rules: How might agents learn or adjust based on feedback or new information?
 - Environment constraints: What are the boundaries and rules of the world they operate in?
 - Feedback loops: How is information from later stages used to improve earlier ones?

Agentic Workflow Visual Modeling

How do we design and communicate these complex systems? The answer is visual representation. We use diagrams to create a map for our workflow, using nodes for tasks and arrows to show the flow of information.

Why Visualize Agent Tasks?

With this agent-centric approach, how do we design and communicate these sometimes complex systems? Visual representation is a very helpful technique.

- **Mapping the Workflow:** Think of it like creating a map. Each task an agent performs, or each distinct stage, is a landmark (a node). Lines or arrows (connectors) show the flow of information or control between these tasks. The aim is a clear picture anyone can understand.
- **Guidelines for Clear Visuals:**
 - **Standard Symbols:** Using consistent symbols (e.g., rectangles for tasks, diamonds for decisions) makes diagrams easier to read.
 - **Clear Labeling:** Each node needs a concise, descriptive name that clearly states its purpose (e.g., "Analyze Customer Feedback" instead of just "Analyze Data").
 - **Show Inputs & Outputs:** It should be obvious what data each task uses and produces. This helps define dependencies and understand the data's journey.
 - **Choose the Right Level of Detail (Granularity):** Sometimes you need a high-level overview (e.g., "Process Order"). Other times, you need to zoom in on sub-tasks (e.g., "Check Inventory," "Process Payment"). The level of detail depends on who you're communicating with and the purpose of the diagram.

Common Types of Agents as Building Blocks

As we start modeling, it's useful to know some common "types" of agents we might use. These are like the different specialists on our AI team. (This isn't an exhaustive list, but a good starting point!)

- **Direct Prompt Agent:** The simplest; sends a user's query directly to an LLM.

- Augmented Prompt Agent: Adds a persona or system instructions to the LLM call to shape the response.
- Knowledge Augmented Prompt Agent: Uses a defined persona AND a specific, curated knowledge base to answer, ignoring the LLM's general knowledge.
- RAG Knowledge Prompt Agent (Retrieval-Augmented Generation): Dynamically retrieves relevant information from a large dataset before answering, making it flexible and less prone to making things up.
- Evaluation Agent: Acts as a quality controller, assessing the output of other agents against criteria and potentially prompting revisions.
- Routing Agent: The "project manager" that directs incoming tasks to the most suitable specialized agent.
- Action Planning Agent: Takes a complex goal and breaks it down into a sequence of smaller, executable steps.

4. Implementing Agentic Workflow Modeling

Demo: Visualizing Workflows

A deterministic workflow for plant maintenance is linear and rigid. Each step follows the last in a predefined sequence, from checking a schedule to inspecting a pump and logging issues. It cannot adapt to new situations.

Visual Notation with Mermaid Diagrams

Mermaid.js is a JavaScript-based tool that lets you create diagrams and charts from text using a syntax similar to Markdown. This means you can write simple, human-readable code to generate complex visuals like flowcharts, sequence diagrams, and Gantt charts.

- Instead of using a mouse to drag and drop shapes, you write text to define the diagram's structure and elements.
- Mermaid provides an online editor where you can write code and see the diagram update in real-time.
- Variety of Diagram Types: Mermaid supports a wide range of diagrams, including:

- Flowcharts
- Sequence diagrams
- Class diagrams
- State diagrams
- Gantt charts
- Pie charts
- Entity-Relationship diagrams
- User Journey diagrams
- Git graphs

5. Agentic Workflow Implementation

Introduction to Implementing Agentic Workflows with Python

The goal is to understand how foundational agent components can be constructed using Python. Think of this as examining the creation of versatile building blocks.

There are different paths you can take:

- A common approach involves using existing agent frameworks, which offer many pre-built components and abstractions.
- Another path is to build the core logic more from scratch, perhaps using Python and a preferred LLM model directly.

This latter approach, while potentially more intensive, offers deep insights and maximum control over the process, allowing you to see exactly how agents can work together. The choice of approach often depends on the project's goals, timelines, and the desired level of customization and understanding. Learning the 'from scratch' principles can be helpful for understanding what's happening inside more complex tools.

Constructing agentic systems using Python

Building dependable systems usually involves several major components, with a focus on crafting the core logic.

1. LLM Models - The Engine: Central to any agent is its access to Large Language Models. These are the engines providing the core intelligence. A flexible system design might allow for different LLMs to be

connected or swapped, making sure of adaptability as new models emerge.

2. **Agent Logic & Prompting - Custom Implementation:** An important layer is the 'agent logic' itself. This is where Python code would define how an agent behaves. This includes crafting effective prompts to communicate with LLMs and implementing the distinct capabilities envisioned for different types of agents.
3. **Workflow Orchestration - Connecting Agents:** With individual agents defined, a system needs to manage how they work together. 'Workflow orchestration' logic, also implemented in Python, would handle the sequence of operations, the flow of information between agents, and the overall execution of multi-agent processes. While various tools can provide these pieces, understanding how to conceptually structure these components in Python offers deep insight into their functionality.

How can you structure Python code to build agentic components?

A design that mirrors a logical separation of concerns can promote modularity and clarity, using standard Python features.

1. **Agent Library/Module:** One common practice is to create a dedicated Python module (e.g., a file named `agent_definitions.py`). This file would house the class definitions for various types of agents. These classes serve as the blueprints defining the agents' attributes and methods, assuming familiarity with Python's object-oriented programming concepts.
2. **Agent Instantiation Logic:** Separately, in a script that drives the workflow, one would import these agent classes. Here, specific instances (objects) of these agent classes would be created and configured. This is where each agent could be customized to its specific role or task.
3. **Workflow Implementation Logic:** Finally, the logic for the actual workflow would be implemented. This involves defining the sequence of tasks, how data is passed between the instantiated agents, and how these agents collaborate to achieve the overall goal of the system.

This structured design—separating agent definitions (typically as classes), instantiation logic (creating objects), and workflow implementation—provides a scalable foundation for developing agentic systems in Python.

6. Agentic Workflow Patterns: prompt chaining workflow

Introduction to prompt chaining Workflow

Prompt Chaining. Get ready to learn how to break down complex tasks into manageable steps for Large Language Models, leading to more accurate and reliable outcomes.

what exactly is Prompt Chaining?

Imagine you have a large, complex task. Trying to get an LLM to solve it in one go can be tricky. Prompt chaining is a technique where we decompose this complex task into a sequence of smaller, more manageable steps. Each step typically involves an LLM call, where the output of one call serves as the input for the next. Why is this useful? By breaking down the problem, we make each individual LLM call simpler.

Prompt chaining is all about task decomposition. Think of it like an assembly line in a factory. Instead of one person trying to build an entire car from scratch, the process is broken down into a series of specialized stations, each performing a specific task.

Similarly, with prompt chaining, we divide a large problem into simpler subtasks. Each LLM call in the chain is like a specialized worker; it only needs to focus on its specific subtask, using the output from the previous 'worker.' This leads to enhanced reasoning across multiple steps and can even reduce LLM 'hallucinations' by keeping the model focused.

While prompt chaining is powerful, it has an inherent challenge. A key characteristic of this pattern is that the output of one LLM call directly informs the input of the next. This creates a dependency chain.

- What happens if an early step in the chain produces a flawed or incomplete output? Unfortunately, this error can propagate and even compound through subsequent steps. Imagine a small mistake in the first calculation of a multi-step financial report. That mistake will carry through, potentially making all following calculations incorrect and ultimately derailing the entire workflow. This risk of error propagation is considerable and highlights an important need we must address.

So, how do we combat error propagation? The answer lies in Intermediate Validation. This involves systematically checking the accuracy, relevance,

and format of the output from each prompt in the sequence before it's passed to the next step. Think of these validation points as 'quality control gates' in our assembly line. Their purpose is to make sure that the intermediate outputs are aligned with what we expect and need for the workflow to succeed. By catching errors early, we prevent them from moving down the chain and being amplified.

Prompt Chaining Strategies

We know we need validation. But how do we actually perform it? There are several distinct approaches, though some may use common implementation techniques:

- **Programmatic Checks:** Here, you write custom code to directly verify specific conditions. For example, checking if an output is valid JSON, if a summary meets a length requirement, or if numerical data falls within an expected range. This is about applying explicit, coded logic.
- **LLM-based Validation:** This approach uses another LLM call to evaluate the output of a previous step. This second LLM can assess more subtle qualities like factual accuracy, relevance to the prompt, or adherence to a specific tone – things that can be hard to codify explicitly.
- **Rule-based Validation:** With this method, you define a set of predefined, explicit rules to check for expected patterns or values. For instance, making sure a generated email contains a salutation and a closing. While these rules are often implemented programmatically, the focus here is on adherence to a specific, declared ruleset.
- **Confidence Scoring:** This method uses the confidence scores that the generating LLM itself might provide for its outputs. These scores are then typically checked programmatically against a set threshold to gauge validity. Outputs below a certain confidence might trigger corrective action. Choosing the right method, or combination of methods, depends on the specific needs of each step in your chain.

So, your validation gate has flagged an incorrect intermediate output. What now? Simply stopping isn't always the best option. We need strategies to handle these situations:

1. **Retry:** The simplest approach. Just re-run the problematic prompt. This might work for transient LLM issues.
2. **Re-prompt with Feedback:** If validation gives specific reasons for failure (e.g., 'summary too long'), incorporate this feedback into a revised prompt. Ask the LLM to regenerate, taking the critique into account. This is key for iterative refinement.
3. **Fallback Mechanisms:** If retries fail, you might terminate the chain and log an error, or route to an alternative, perhaps simpler, task or a pre-defined 'safe' output.
4. **Simplified Critique and Refinement:** An LLM can be prompted to critique its own previous output based on given criteria. Then, another prompt instructs it to refine the output based on that critique. Don't forget Logging and Monitoring! Comprehensive logs of inputs, outputs, and validation results at each step are very important for debugging and understanding where and why a chain might be failing.

Effective management of context is also important in prompt chaining. While each step needs enough information from previous outputs, overloading an LLM with excessive or irrelevant context can degrade performance or exceed context window limits. It's a balancing act. We need **Selective Context Passing**: Only the most relevant parts of previous outputs should be passed as context. Plan what information each step truly needs. Prompt chaining itself helps manage finite context limits by distributing information across prompts. Sometimes, to prevent the LLM from 'forgetting' important earlier details, it's useful to use **Contextual Reiteration** – restating important context within each new prompt. The goal is to provide precisely the necessary and sufficient context for each step, making sure the LLM stays focused and performs effectively.

8. Agentic Workflow Patterns: Routing

Agentic Workflow Patterns: Routing

Imagine you have a complex project with many different kinds of tasks. Some are simple, some require deep expertise, and others need specific tools. How do you make sure each task goes to the right place or person efficiently?

This is where the **'Routing' pattern** in agentic workflows comes into play. This pattern is designed to intelligently direct incoming tasks or inputs – whether they're raw user requests or steps from a planning agent – to different processing paths or specialized agents based on the nature of the input itself.

So, why is routing so beneficial in agentic systems? Think of it like a sophisticated mail sorting facility. Instead of one person trying to handle every type of mail, specialized systems and routes make sure each piece gets to its destination quickly and correctly. Routing offers several key advantages:

It enables **task specialization**. By directing inputs to agents or prompts highly optimized for specific tasks, we achieve better performance and accuracy.

Routing is a powerful tool for **resource optimization**. We can direct tasks to different LLMs based on complexity, cost, and latency requirements. Simpler tasks might use faster, cheaper models, while complex ones go to more powerful models.

Routing provides **flexibility and adaptability**. The system can handle diverse types of requests by dynamically choosing the right path for each one.

It also enhances **scalability**, as new specialized agents or processing paths can be added easily as needs evolve.

At the heart of the routing pattern are two fundamental stages: **Classification** and **Task Dispatch**.

Imagine you're managing customer support. First, you need to understand what each incoming query is about – is it a billing question, a technical issue, or a sales inquiry? That's **Classification**.

Classification is the initial step where the system analyzes an incoming task or input to determine its type, category, intent, or even its complexity. The goal is to understand the nature of the input so an informed decision can be made about how to handle it.

Once the input is classified, the next stage is **Task Dispatch**.

Based on the classification, the workflow directs or 'dispatches' the input (and its classification label) to the appropriate specialized agent, a specific prompt chain, a function, or a dedicated processing module.

This is like sending that customer query to the billing department, the tech support team, or the sales team. Together, these two stages make sure that tasks are not just processed, but are processed by the most suitable component in the system.

Let's focus on the first stage: **Classification**. The primary goal here is to accurately determine the nature of the incoming input. What is this task asking for? What category does it fall into? What is the user's underlying intent?

Effective classification is key because the accuracy of the entire routing process depends on it. If a task is misclassified, it might be sent to the wrong agent, leading to inefficiency or incorrect results.

To perform this classification, systems can use various methods.

Rule-based classification uses predefined programmatic logic, like keyword matching or conditional statements.

LLM-based classification uses a Large Language Model to analyze the input and assign it to a category based on contextual understanding.

The choice of method depends on factors like input complexity, desired accuracy, and performance needs. For now, the key is to understand that classification *happens* to prepare the input for the next stage.

Once an input has been successfully classified, we move to the second core stage of routing: **Task Dispatch**. Task Dispatch is the process of taking the classified input and directing it to the most appropriate agent, system, or workflow designed to handle that specific category of task.

This is where the actual 'routing' and **branching logic** comes into play. Think of it like this:

The classification stage told us *what* the task is (e.g., 'Sales Inquiry').

The task dispatch stage decides *where* it goes based on that classification (e.g., to the 'Sales Team Agent'). This implements branching: If the input is classified as 'Type A', it's dispatched down Path A to Worker Agent A. If classified as 'Type B', it's dispatched down Path B to Worker Agent B.

The 'appropriateness' of a worker agent or processing path is determined by its designated expertise or capability, which aligns with the categories from the classification stage. This makes sure that each task is handled by

the component best equipped for it, fulfilling the intelligent distribution aspect of routing.

10. Agentic Workflow Patterns : Parallelizations

Ever felt like a complex task is taking forever? What if you could divide and conquer it, not just to save time, but also to get a better outcome? That's where the **power of parallelization in agentic workflows** comes in. This is a technique where multiple agents work on different parts of a task, or even the same task, at the same time.

Think of parallelization like a team project. Instead of one person doing everything sequentially, you split the work among team members who can tackle their parts simultaneously.

In agentic workflows, these 'team members' are AI agents. The idea is simple: distribute the workload, process in parallel, and then consolidate the results. This is much like the 'scatter-gather' pattern you might see in distributed computing, where a problem is 'scattered' to multiple workers and their individual findings are 'gathered'. This process generally involves a way to

- split the main task
- individual agents perform the sub-tasks
- a method to aggregate their outputs into a final, coherent result.

And remember, it's not just about speed. By having multiple agents, we can also generate diverse perspectives or solutions, leading to potentially richer or more dependable outcomes.

Before we jump into how to split tasks, there's a fundamental prerequisite, a golden rule for effective parallelization: subtasks **must** be largely independent of one another. What does this mean? It means that **Agent A working on Subtask A shouldn't need to wait for Agent B to finish Subtask B to do its own work**. If the output of one subtask is a direct input for another, then a sequential pattern, like prompt chaining, might be more suitable for those dependent parts.

Effective parallelization starts with smart task decomposition. Let's briefly cover some key strategies:

- **Sectioning or Sharding:** This is for large, divisible inputs. You split the input, like a long document or a large dataset, into smaller chunks, and each chunk is processed in parallel. For instance, each chapter of a book could be summarized by a different agent at the same time.
- **Aspect-Based Decomposition:** Use this when a task requires analyzing multiple independent aspects of a single subject. Different agents can concurrently investigate distinct facets – like one agent checking technical specs, another user sentiment, and a third competitor pricing for a product review.
- **Identical Tasks for Diversity or Voting:** Here, you run the same core task multiple times in parallel, perhaps with varied prompts or models. This is great for generating a range of creative outputs or for achieving a more reliable result by having multiple agents 'vote' on an answer, enhancing overall quality.

Once parallel tasks are complete, we need to combine their outputs. Here are common aggregation strategies:

- **Concatenation:** This is the simplest – you just join the outputs from parallel tasks together, like appending individual chapter summaries to form a complete document.
- **Comparison and Selection:** If you've generated multiple solutions to the same problem in parallel, this method involves evaluating them based on predefined criteria and then selecting the 'best' one.
- **Voting/Majority Rule:** This is a powerful consensus mechanism. When multiple agents independently perform the same classification or answer a question, the most frequently occurring output is chosen as the final answer, boosting accuracy.
- **Synthesizer LLM or Aggregator LLM:** A dedicated LLM can take multiple, potentially diverse outputs from the parallel tasks and skillfully weave them into a single, coherent, and often more subtle final response.

12. Agentic Workflow patterns: Evaluator-Optimizer workflow

Imagine you've tasked an AI agent with a task that requires a high-quality output, like drafting a comprehensive research report, generating critical

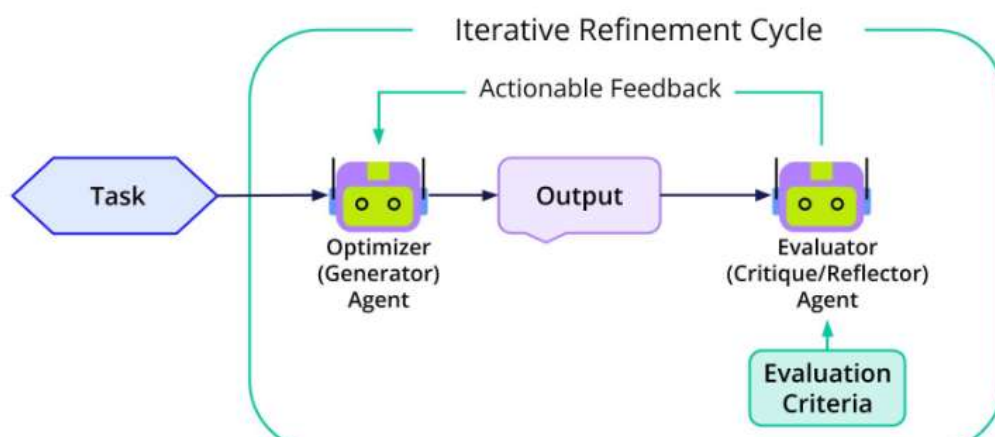
software code, or creating a detailed marketing plan. The initial output is a good start, but how can you be absolutely sure it's accurate, complete, adheres to all constraints, and truly meets the high standards required? This is a common challenge in working with AI agents.

We're going to explore a powerful agentic workflow pattern specifically designed to address this: the **Evaluator-Optimizer workflow**. You'll learn how this pattern uses an iterative cycle of generation, critique, and refinement to significantly improve the quality and reliability of outputs from AI agents.

At its core, the evaluator-optimizer workflow has two key roles:

1. The **Optimizer (or Generator) Agent**: This agent takes the initial task and *generates* an output. It also *refines* this output based on guidance it receives.
2. The **Evaluator (or Critique/Reflector) Agent**: This agent acts like an expert reviewer. It assesses the Optimizer's output against predefined Evaluation Criteria – these are the standards for success. Based on this assessment, it provides specific, Actionable Feedback. This cycle – where the Optimizer generates, the Evaluator critiques against criteria, and the Optimizer refines based on that feedback – then iterates. With each loop, the output ideally gets closer to the desired quality.

The Evaluator-Optimizer Cycle



To make the iterative loop effective, several elements are important:

- **Clear Evaluation Criteria:** You must precisely define what constitutes a successful output. Are your criteria specific? Can they be measured or objectively checked? Are they unambiguous? These criteria form the bedrock of the Evaluator's assessment and guide the entire refinement process.
- **Actionable Feedback:** The feedback provided by the Evaluator to the Optimizer is very important. It can't be vague like 'do better.' It needs to be specific, constructive, and provide enough detail for the Optimizer to understand *what* needs to change and ideally *how* to make that change. The Optimizer then uses this feedback, along with the original task instructions and its previous attempt, to generate an improved version.
- **Stopping Conditions:** An iterative loop needs a defined end. You must specify when the process should conclude. Common stopping conditions include:
 - the output finally meets all evaluation criteria
 - a predetermined maximum number of iterations has been reached
 - The improvement between cycles becomes negligible (diminishing returns). This makes sure the process is both effective and finite.
 - Getting these three elements right is key to using the full power of the Evaluator-Optimizer workflow.

By establishing clear roles for an **Optimizer** to generate and refine, and an **Evaluator** to critique based on set standards, we create a cycle of continuous improvement.

This iterative refinement approach is invaluable for tasks demanding high precision, adherence to complex requirements, and overall top-tier quality.

14. Agentic Workflow Patterns : Orchestrator-Workers Workflow

Let's explore one of the most powerful and flexible: the Orchestrator-Workers pattern.

Imagine needing to solve a complex problem that's too big for any single AI agent, or where the steps aren't clear from the start. How would you coordinate a team of specialized AI assistants to tackle it effectively? The Orchestrator-Workers workflow is a sophisticated multi-agent system design.

Think of the Orchestrator-Workers pattern like a skilled project manager (the **Orchestrator**) leading a team of expert contractors (the **Worker Agents**). The project manager understands the big project, breaks it down, assigns tasks to the right experts, and then assembles their contributions. At its heart, this pattern involves two main roles:

- **The Orchestrator Agent:** The main coordinating agent. It analyzes a complex task, dynamically breaks it into subtasks, and delegates these to Worker agents.
- **Worker Agents:** Specialized agents, each skilled in a particular function (e.g., research, analysis, writing), executing subtasks assigned by the Orchestrator. The Orchestrator then synthesizes the outputs from these Workers to produce the final solution.

What does the Orchestrator actually do?

Its primary role is orchestration: taking a complex goal, dynamically breaking it down into logical sub-tasks at runtime, and then intelligently delegating these sub-tasks to the most suitable specialized Worker agents.

Let's make this concrete with an example of a 'Market Analysis Report' for electric vehicles in Europe. When the Orchestrator receives this request, it doesn't just follow a fixed checklist. It analyzes the request and decides, on the fly, that it needs several pieces of work done.

- it might determine the need for: A 'News Collection' sub-task, assigned to a Web Research Worker;
- a 'Competitor Strategy Analysis' sub-task, given to a Data Analysis Worker;

- and a 'Market Trend Identification' sub-task, delegated to a Trend Spotting Worker.

This ability to dynamically decide and delegate the 'what' and 'who' based on the specific request is the core of its orchestration power.

Once the Worker agents have done their jobs, they send their individual findings back to the Orchestrator. Now, the Orchestrator has another very important role: synthesis. Synthesis means taking all these separate pieces of information and combining them into a single, coherent, and useful final output. It's more than just stapling pages together; it's about connecting diverse inputs to create new understanding or a complete product. The orchestrator is responsible for making sense of all the partial results and forming a holistic response.

Let's see how synthesis works with our Market Analysis Report, continuing from where our workers completed their tasks.

- The Orchestrator has now received: recent industry news from the Web Search Worker,
- competitor strategy analysis from the Analysis Worker,
- and key market trends from the Trend Spotting Worker.

Now, the Orchestrator must weave these distinct pieces into a cohesive report. This might involve summarizing key data points, structuring the information with an introduction, body sections for news, competitors, and trends, and a concluding outlook. The goal is to produce a comprehensive, well-organized document that directly answers the original request, making sense of all the gathered information. This transformation of partial inputs into a final, polished product is the core of synthesis in this context.

You might wonder how this sophisticated Orchestrator-Workers pattern differs from simply running tasks in parallel. The core distinction lies in its dynamic intelligence and flexibility.

- **Orchestrator-Workers:** This pattern is like having a smart project manager.

- The Orchestrator analyzes the problem at runtime, dynamically decides on the necessary sub-tasks, and assigns them to the best-suited specialist workers.
- It's highly flexible and excels at tackling complex, unpredictable problems where the solution path isn't known in advance.
- The orchestrator actively manages, delegates, and synthesizes information.
- **Simple Parallelization:** This is more akin to an assembly line.
 - Tasks are typically pre-defined, and the workflow is static – it breaks down a job into fixed, known parts that can be processed simultaneously.
 - It's very efficient for repetitive work that can be clearly divided into independent chunks, but it lacks the adaptability for novel or evolving requirements.

Orchestrator-Workers brings dynamic intelligence and adaptability for challenges, while simpler parallelization is about efficiently processing predictable, divisible workloads.

-